Introduction to ggcpb

- 1 What is ggcpb?
  - 1.1 Why ggcpb instead of James?
  - 1.2 What you need to know to use ggcpb in Excel
  - 1.3 Use ggcpb in R
- 2 Figure types
  - 2.1 Line with forecast
  - 2.2 Columns with two axes
  - 2.3 Horizontal bars with chosen colours
  - 2.4 Line with markers in English
  - 2.5 Boxplot
  - 2.6 Extended boxplot on a full page
  - 2.7 Donut chart
  - 2.8 Donut chart with leader labels
  - 2.9 Dot plot
  - 2.10 Scatter plot
  - 2.11 Histogram
  - 2.12 Classed map
- 3 Ggplot2 base allows for endless possibilities
- 4 Settings for the whole publication
- 5 Example code to create figures in R
  - 5.1 Line with forecast
  - 5.2 Columns with two axes
  - 5.3 Horizontal bars with chosen colours
  - 5.4 Line with markers in English
  - 5.5 Boxplot
  - 5.6 Extended boxplot on a full page
  - 5.7 Donut
  - 5.8 Donut chart with leader labels
  - 5.9 Dot plot
  - 5.10 Scatter plot
  - 5.11 Histogram
  - 5.12 Classed map

This vignette gives a short introduction to creating customizable CPB style figures with ggcpb.

1 What is ggcpb?

ggcpb is the new version of James. It draws figures in the CPB house style from Excel or from R.

1.1 Why ggcpb instead of James?

ggcpb is more future proof. It is built on ggplot2, the most widely used plotting package in R. This brings the following benefits:

- **More possibilities.** Anything ggplot2 can do, ggcpb can do. If you need an extra layer or a special annotation, you can add it. It allows for new figure types that were not possible with James, such as a donut chart.
- **Easier to maintain.** ggcpb adds a thin layer to a well tested package.

1.2 What you need to know to use ggcpb in Excel

This is the general workflow to create figures with ggcpb without using R:

1. Find the most recent ggcpb version.
2. Copy it to the place where you want to create figures.
3. Put your datasets in the folder `data`. Open `params.csv` in Excel and describe your figures.
4. Start `generate_plot.bat` by double clicking it. This runs the `generate_plot.R` script. Your figures appear in the folder `generated`. Each image is named after its dataset. Running the script again produces updated figures. Images with the same name are replaced. A dataset or figure that fails is skipped and listed at the end, and in `params_log.txt`. The other figures are still saved.

Please note. The `generate_plot.bat`, `generate_plot.R` and `params.csv` must all be in the same folder. Within this folder, there must be a subfolder `data`, in which each dataset must be saved as a CSV file, and *not* as spreadsheet (`xlsx`). In Excel, choose `File` -> `Save as` -> `CSV UTF-8 (comma delimited)`.

You do not need to know R to follow this workflow. One parameter file and one or more datasets drive everything.

- `params.csv` describes your figures. Column 1 holds the parameter names, such as `title`, `x` and `data`. Every other column describes one figure. The row `data` names the dataset of the figure. All parameters and their descriptions are given in the import vignette.
- The folder `data` holds your datasets. Each dataset is its own CSV file with any name, such as `koopkracht.csv`. The first row holds the column names. The following rows are the observations per column.

Add the row `page` to save a figure at `full` width (15.5 cm). The default is `half` (7.5 cm). Write the dataset name without `.csv`, such as `koopkracht`. Leave `data` blank to use `data/data.csv`. Write `a;b` to draw the same figure for both datasets. A figure is saved under the name of its dataset, so `koopkracht` gives `koopkracht.png`. If two figures use the same dataset, the figure number is added, such as `koopkracht_1.png`.

Run the script again after every change. This will update the figures. The import vignette gives a more detailed explanation and lists all settings per figure type. See `vignette("import")`.

1.3 Use ggcpb in R

Next to plotting Excel data with ggcpb, you can also plot data that you already have in R. The caption below each figure in this introduction shows the function to use. Load the packages first.

```
library(conflicted) # optional, but recommended
library(ggcpb)
```

Each figure type has its own function.

```
# Line chart
cpb_line(data = df, x = year, y = value)
# Bar chart
cpb_col(data = df, x = year, y = value, fill = group)
# Area chart
cpb_area(data = df, x = year, y = value, fill = group)
```

```r
# Box plot
cpb_box(data = df, x = group, p5 = p5, p25 = p25, p50 = p50, p75 = p75, p95 = p95)
# Dot plot
cpb_dot(data = df, x = term, y = estimate, lower = lower, upper = upper)
# Donut chart
cpb_donut(data = df, fill = group, y = share)
# Scatter plot
cpb_scatter(data = df, x = income, y = spending)
# Histogram
cpb_hist(data = df, x = duration)
# Map
cpb_map(data = df, region = code, value = share)
```

To export a figure, use `save_cpb()`. It writes the figure at the CPB page widths. Use `page = "half"` for 7.5 cm and `page = "full"` for 15.5 cm.

```r
save_cpb("zonnepanelen.png", p, page = "half")
```

The rest of this introduction assumes that you work in Excel.

2 Figure types

Each figure below is combined with a feature that you can also use with several other plot types, if not all. The parameters and their corresponding values are portrayed next to the figure. Other settings keep their default.

The style of the figures can be set for a whole publication. See section 4.

2.1 Line with forecast

A line chart with a forecast window and an uncertainty band.



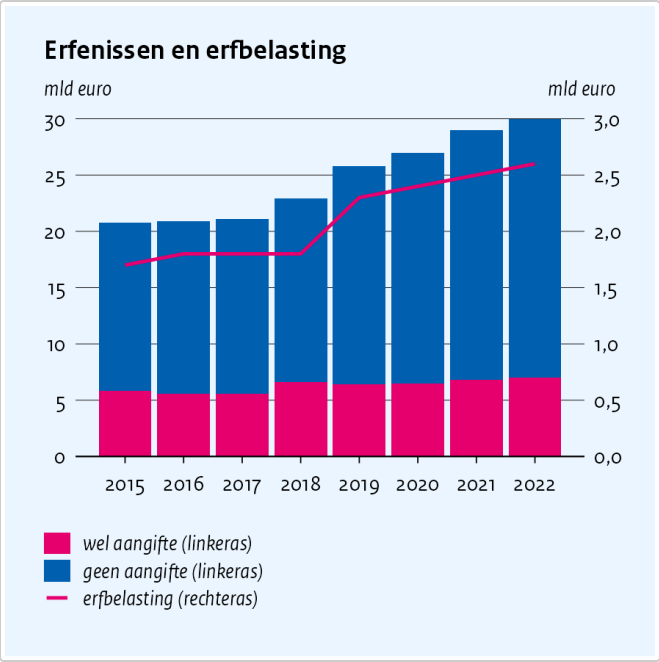
Relevant parameters

plot_type	line
title	Economische groei met onzekerheid
x	jaar
y	groei
ymin	lo
ymax	hi
forecast_x	2023.5
ylab	%

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `line-forecast.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_line()` instead. Follow the example code in section 5.1 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_line()`.

2.2 Columns with two axes

Two series on one figure, with the second series on the right axis. The legend tells you which axis belongs to which series.



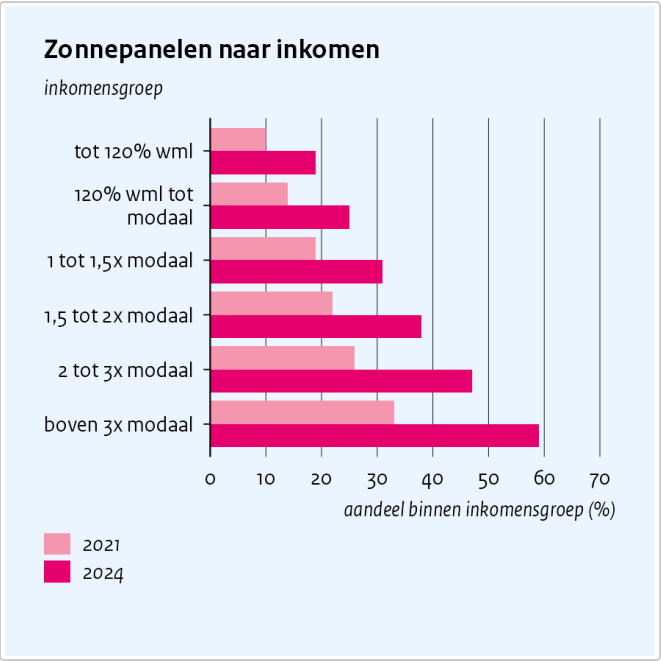
Relevant parameters

plot_type	col
title	Erfenissen en erfbelasting
x	jaar
y	mld
fill	soort
sec_y	heffing
sec_limits	0;3
sec_legend_label	erfbelasting
sec_ylab	mld euro
ylab	mld euro

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `col-secaxis.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_col()` instead. Follow the example code in section 5.2 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_col()`.

2.3 Horizontal bars with chosen colours

The colours decide the legend. Pick them with `colour_index`.



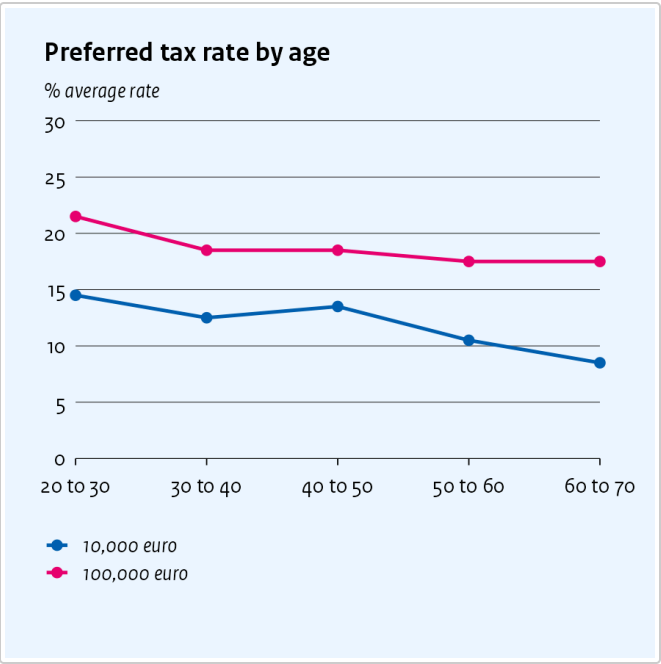
Relevant parameters

plot_type	col
title	Zonnepanelen naar inkomen
x	groep
y	aandeel
fill	jaar
position	dodge
orientation	horizontal
colour_index	2;1
value_limits	0;70
ylab	inkomensgroep
xlab	aandeel binnen inkomensgroep (%)

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `col-horizontal.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_col()` instead. Follow the example code in section 5.3 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_col()`.

2.4 Line with markers in English

A line with markers suits a few ordered categories. This figure uses the English style.



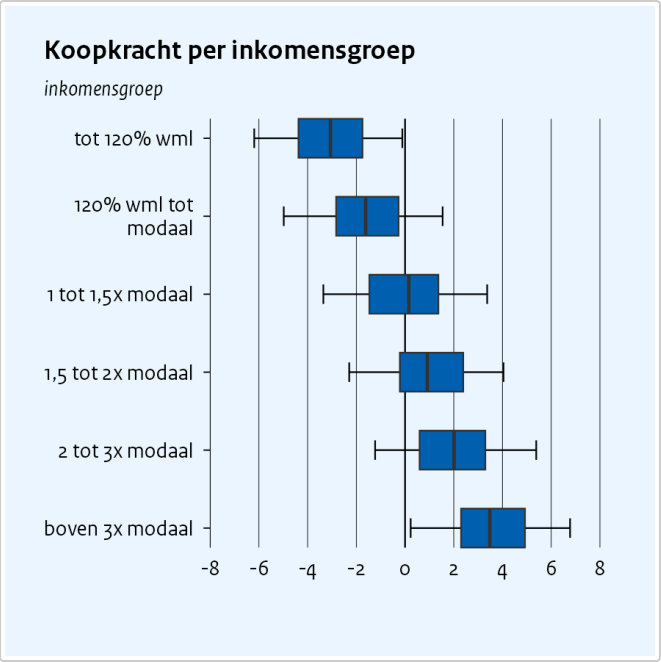
Relevant parameters

plot_type	line
title	Preferred tax rate by age
x	leeftijd
y	tarief
colour	erfenis
points	TRUE
value_limits	0;30
style	english
ylab	% average rate

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `line-points.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_line()` instead. Follow the example code in section 5.4 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_line()`.

2.5 Boxplot

A box shows the 25th to 75th percentile. The line in the box is the median. The whiskers run from the 5th to the 95th percentile.



Relevant parameters

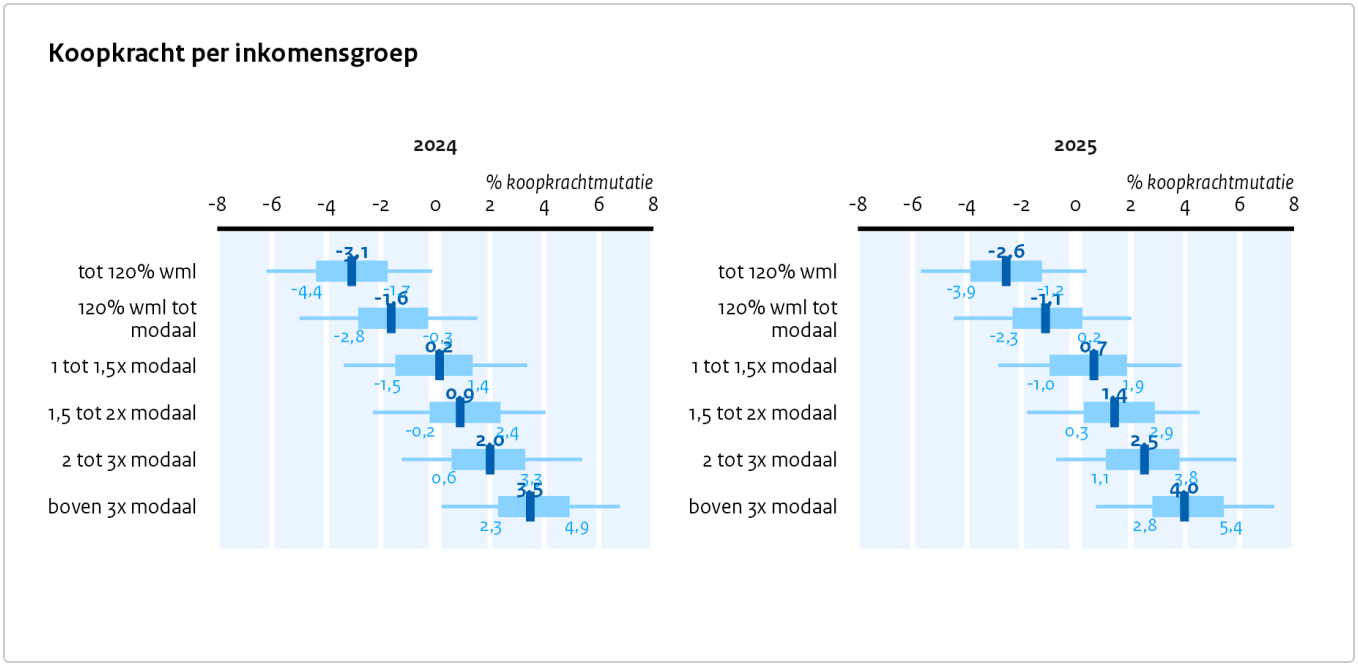
plot_type	box
title	Koopkracht per inkomensgroep
x	groep
p5	p5
p25	p25
p50	p50
p75	p75
p95	p95
orientation	horizontal
subtitle	inkomensgroep
ylab	% koopkrachtmutatie

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `box.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_box()` instead. Follow the example code in section 5.5 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_box()`.

More box plot styles are in `vignette("boxplots")`.

2.6 Extended boxplot on a full page

The extended boxplot has the CPB ramingen-style. Through the `facet` parameter the data gets grouped into the year column, which results in one panel per year in the dataset. The default figure is saved at the full page width (15.5 cm) with `page = full`.



Relevant parameters

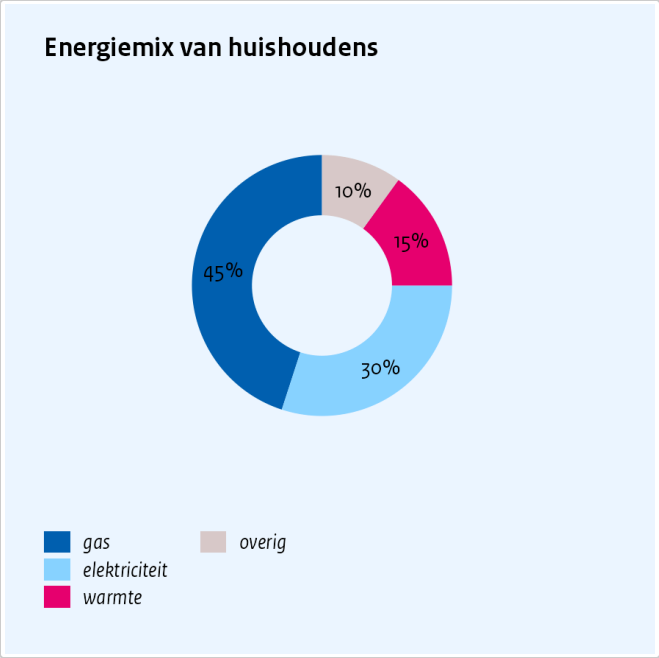
plot_type	boxplot_extended
title	Koopkracht per inkomensgroep
x	groep
p5	p5
p25	p25
p50	p50
p75	p75
p95	p95
facet	jaar
facet_ncol	2
ylab	% koopkrachtmutatie
page	full

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `box-extended.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_boxplot_extended()` instead. Follow the example code in section 5.6 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_boxplot_extended()`.

2.7 Donut chart

Individual categories add up to 100% in the donut chart. Categories don't need to be expressed as

percentages. Any numerical values will work, as the donut chart automatically calculates the proportions.



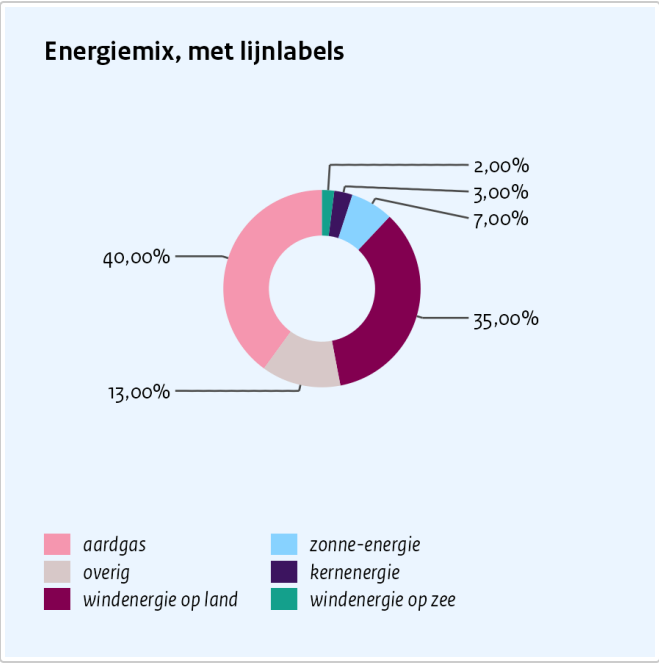
Relevant parameters

plot_type	donut
title	Energiemix van huishoudens
fill	bron
y	aandeel
colour_index	6;5;2;4

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `donut.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_donut()` instead. Follow the example code in section 5.7 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_donut()`.

2.8 Donut chart with leader labels

A donut can also print its labels outside the ring, on leader lines. This one uses more categories, two decimals in the percentages and colours from the extended palette.



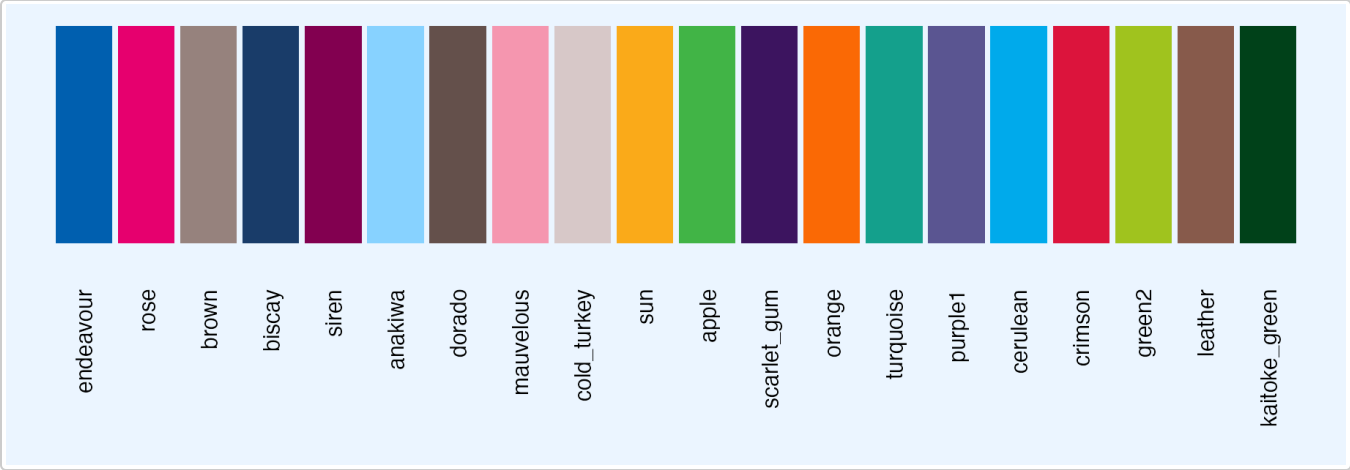
Relevant parameters

plot_type	donut
title	Energiemix, met lijnlabels
fill	bron
y	share
label_style	leader
leader_length	0.08
label_accuracy	0.01
palette	extended
colour_index	8;9;5;6;12;14;15

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `donut-leader.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you

can call `cpb_donut()` instead. Follow the example code in section 5.8 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_donut()`.

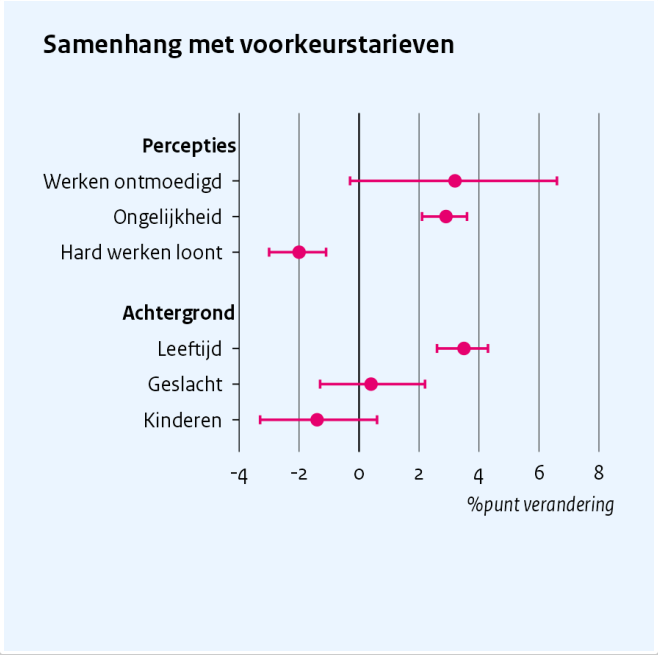
This donut uses the extended palette, which consists out of the following 20 distinct colours:



2.9 Dot plot

A dot plot shows an estimate with its interval.

With `legend = filled-empty` the figure has no legend, but it keeps the space of one. This gives the same panel height as a figure with a legend.



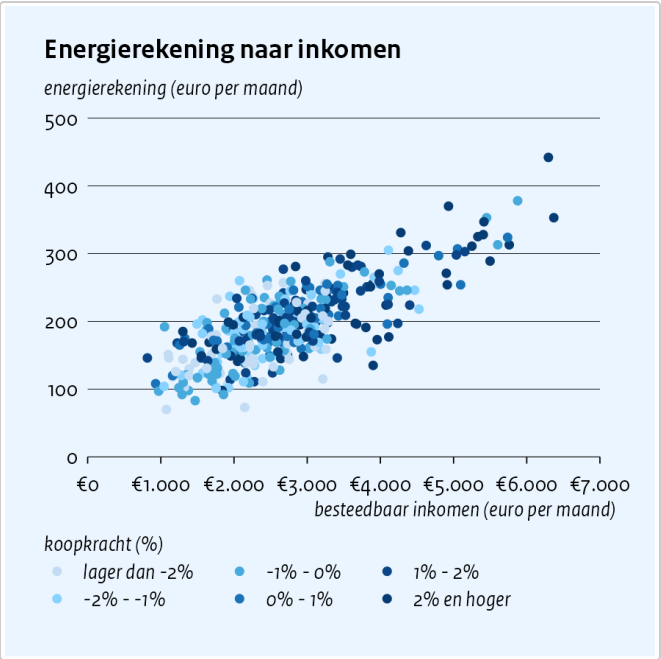
Relevant parameters

plot_type	dot
title	Samenhang met voorkeurstarieven
x	term
y	coef
lower	lo
upper	hi
group	blok
legend	filled-empty
xlab	%punt verandering

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `dot.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_dot()` instead. Follow the example code in section 5.9 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_dot()`.

2.10 Scatter plot

The parameter `cut` puts the purchasing power change into classes. The classes give the points their colour.

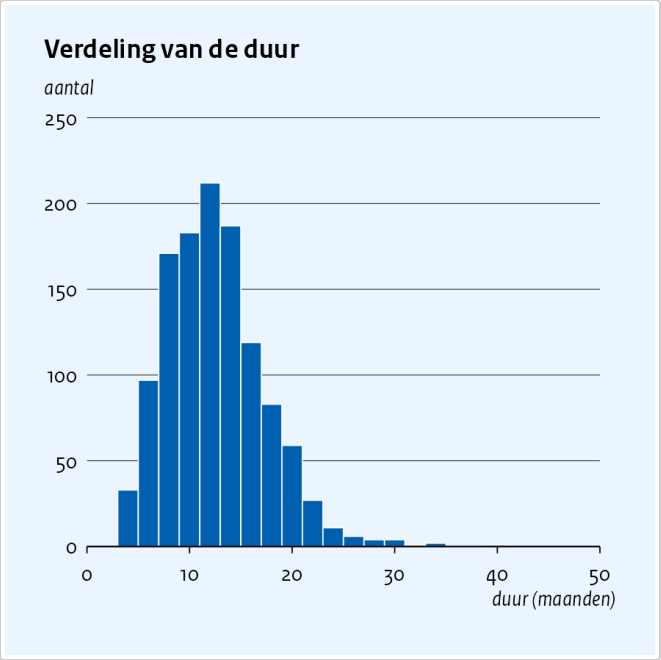


Relevant parameters

plot_type	scatter
title	Energierekening naar inkomen
x	inkomen
y	energierekening
colour	koopkracht
cut	-Inf;-2;-1;0;1;2;Inf
cut_labels	pct
palette	blues
legend_ncol	2
ylab	energierekening (euro per maand)
xlab	besteedbaar inkomen (euro per maand)
x_labels	euro
legend_title	koopkracht (%)

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `scatter.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_scatter()` instead. Follow the example code in section 5.10 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_scatter()`.

2.11 Histogram



Relevant parameters

plot_type	hist
title	Verdeling van de duur
x	maanden
binwidth	2
ylab	aantal
xlab	duur (maanden)

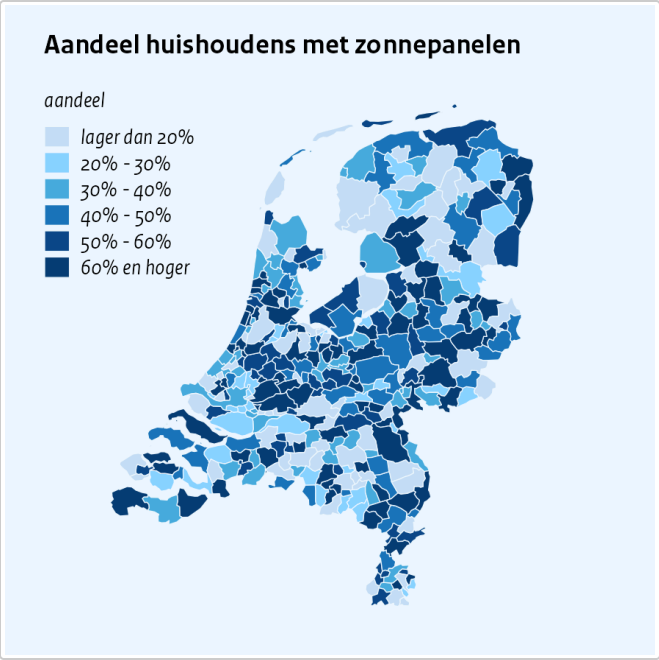
How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `hist.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call

`cpb_hist()` instead. Follow the example code in section 5.11 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_hist()`.

2.12 Classed map

The maps can display municipalities or provinces on bundled generalised CBS/Kadaster boundaries. The following map colours each Dutch municipality by class. The parameter `cut` makes the classes, and `cut_labels = pct` writes them as percentages. In R, `cpb_cut()` and `label_pct_nl()` do the same. The boundaries come with the package.

```
gemeenten <- tibble(code = unique(cpb_nl_geo("gemeente")$code)) |>
  mutate(aandeel = runif(n(), 5, 75),
         klasse  = cpb_cut(aandeel, breaks = c(0, 20, 30, 40, 50, 60, Inf),
                           labeller = label_pct_nl()))
```



Relevant parameters

plot_type	map
title	Aandeel huishoudens met zonnepanelen
region	code
value	aandeel
cut	0;20;30;40;50;60;Inf
cut_labels	pct
palette	blues
legend_title	aandeel

How to reproduce this figure: copy `params.csv`, `generate_plot.bat` and the folder `data` with `map.csv` to a personal folder. Fill in the parameters above in `params.csv` and start the script. In R you can call `cpb_map()` instead. Follow the example code in section 5.12 to create the plot in R. Alternatively, the csv file can be read with `read.csv()` and used as input to `cpb_map()`.

More options are in `vignette("maps")`.

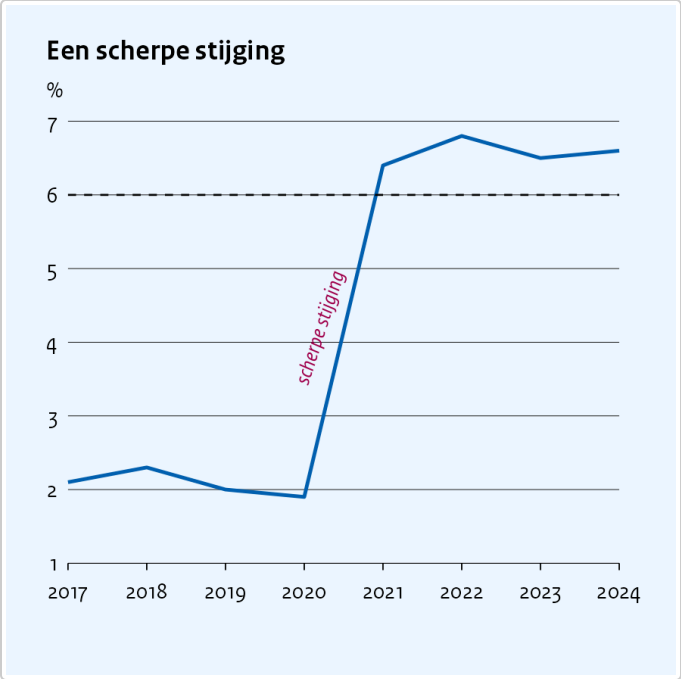
3 Ggplot2 base allows for endless possibilities

Every ggcpb figure is a real ggplot2 object. This means you can stack extra functionalities to the plot in R with `+`.

The next figure has a sharp rise between 2020 and 2021. We add a reference line, and a text along the diagonal in dark pink italic letters. The position comes from the coordinates of the data. The `angle` follows the slope of the line.

```
stijging <- tibble(jaar = 2017:2024,
                   waarde = c(2.1, 2.3, 2.0, 1.9, 6.4, 6.8, 6.5, 6.6))

cpb_line(stijging, x = jaar, y = waarde,
         title = "Een scherpe stijging", ylab = "%") +
  geom_hline(yintercept = 6, linetype = "dashed", linewidth = 0.3) +
  annotate("text", x = 2020.2, y = 4.2, label = "scherpe stijging",
           angle = 72, colour = "#a0004e", fontface = "italic",
           size = 2.4, family = cpb_font_family())
```



You can stack as many pieces as you need. More patterns are in `vignette("annotation")`.

4 Settings for the whole publication

Some settings are the same for every figure of a publication. The main one is the `style`. Dutch is the default. An English report uses `english`. It sets the number format and the words, such as the forecast label.

In R, set it once at the start of the script. Every figure then uses it, unless a figure sets its own `style`.

```
options(ggcpb.style = "dutch") # or "english"
```

In Excel there is no single setting for all figures. Add the row `style` to `params.csv` and give every figure the same value in its column, such as `english`. A blank cell uses the Dutch default.

5 Example code to create figures in R

The code below creates the figures above in R. It first builds the data, then draws the figure. It is the same code that made the figures in this document. Start the script with these lines.

```
library(conflicted)
library(ggcpb)
library(ggplot2)
library(dplyr)
library(tidyr)

options(ggcpb.style = "dutch") # style of the whole publication
```

5.1 Line with forecast

```
groei <- tibble(jaar = 2015:2027,
                groei = round(rnorm(13, 1.5, 0.8), 1)) |>
  mutate(marge = c(rep(0, 9), 0.4, 0.9, 1.4, 1.8),
         lo = groei - marge, hi = groei + marge)
cpb_line(groei, x = jaar, y = groei, ymin = lo, ymax = hi,
         forecast_x = 2023.5,
         title = "Economische groei met onzekerheid", ylab = "%") +
  scale_x_continuous(breaks = seq(2015, 2027, 3), minor_breaks = 2015:2027,
                     guide = guide_axis(minor.ticks = TRUE))
```

5.2 Columns with two axes

```
erfenissen <- expand_grid(
  soort = factor(c("geen aangifte", "wel aangifte"),
                 levels = c("geen aangifte", "wel aangifte")),
  jaar  = 2015:2022
) |>
  mutate(mld = c(15.0, 15.3, 15.5, 16.3, 19.4, 20.5, 22.2, 23.0,
                 5.8, 5.6, 5.6, 6.6, 6.4, 6.5, 6.8, 7.0)) |>
  left_join(tibble(jaar = 2015:2022,
                   heffing = c(1.7, 1.8, 1.8, 1.8, 2.3, 2.4, 2.5, 2.6)),
            by = "jaar")
cpb_col(erfenissen, x = jaar, y = mld, fill = soort,
        sec_y = heffing, sec_limits = c(0, 3),
        sec_legend_label = "erfbelasting", sec_ylab = "mld euro",
        title = "Erfenissen en erfbelasting", ylab = "mld euro")
```

5.3 Horizontal bars with chosen colours

```
groepen <- c("tot 120% wml", "120% wml tot modaal", "1 tot 1,5x modaal",
             "1,5 tot 2x modaal", "2 tot 3x modaal", "boven 3x modaal")
zonnepanelen <- expand_grid(
  jaar  = factor(c(2021, 2024), levels = c(2024, 2021)),
  groep = factor(groepen, levels = rev(groepen))
) |>
  mutate(aandeel = c(10, 14, 19, 22, 26, 33,
```

```
                     19, 25, 31, 38, 47, 59))
cpb_col(zonnepanelen, x = groep, y = aandeel, fill = jaar,
        position = "dodge", orientation = "horizontal",
        colour_index = c(2, 1), value_limits = c(0, 70), width = 0.85,
        title = "Zonnepanelen naar inkomen",
        ylab  = "inkomensgroep", xlab = "aandeel binnen inkomensgroep (%)")
```

5.4 Line with markers in English

```
leeftijden <- c("20 to 30", "30 to 40", "40 to 50", "50 to 60", "60 to 70")
tarieven <- expand_grid(
  erfenis  = factor(c("10,000 euro", "100,000 euro"),
                    levels = c("10,000 euro", "100,000 euro")),
  leeftijd = factor(leeftijden, levels = leeftijden)
) |>
  mutate(tarief = c(14.5, 12.5, 13.5, 10.5, 8.5,
                    21.5, 18.5, 18.5, 17.5, 17.5))
cpb_line(tarieven, x = leeftijd, y = tarief, colour = erfenis,
         points = TRUE, value_limits = c(0, 30), style = "english",
         title = "Preferred tax rate by age", ylab = "% average rate")
```

5.5 Boxplot

```
raw <- tibble(
  groep      = factor(rep(groepen, each = 400), levels = rev(groepen)),
  koopkracht = rnorm(2400, mean = rep(c(-3, -1.5, 0, 1, 2, 3.5), each = 400),
                     sd = 2)
)
kk <- raw |>
  summarise(p5 = quantile(koopkracht, 0.05), p25 = quantile(koopkracht, 0.25),
            p50 = quantile(koopkracht, 0.50), p75 = quantile(koopkracht, 0.75),
            p95 = quantile(koopkracht, 0.95), .by = groep)
cpb_box(kk, x = groep,
        p5 = p5, p25 = p25, p50 = p50, p75 = p75, p95 = p95,
        orientation = "horizontal",
        title = "Koopkracht per inkomensgroep",
        subtitle = "inkomensgroep", ylab = "% koopkrachtmutatie")
```

5.6 Extended boxplot on a full page

```
kk_jaar <- bind_rows(
  mutate(kk, jaar = "2024"),
  mutate(kk, jaar = "2025", across(p5:p95, \(v) v + 0.5))
)
cpb_boxplot_extended(kk_jaar, x = groep,
                     p5 = p5, p25 = p25, p50 = p50, p75 = p75, p95 = p95,
                     facet = jaar, facet_ncol = 2,
```

```
                    title = "Koopkracht per inkomensgroep",
                    ylab = "% koopkrachtmutatie")
```

5.7 Donut

```
bronnen <- c("gas", "elektriciteit", "warmte", "overig")
energie <- tibble(bron = factor(bronnen, levels = bronnen),
                  aandeel = c(45, 30, 15, 10))
cpb_donut(energie, fill = bron, y = aandeel,
          colour_index = c(6, 5, 2, 4),
          title = "Energiemix van huishoudens")
```

5.8 Donut chart with leader labels

```
niveaus <- c("aardgas", "overig", "windenergie op land", "zonne-energie",
             "kernenergie", "windenergie op zee")
energie2 <- tibble(
  bron  = factor(c("aardgas", "windenergie op land", "overig",
                   "kernenergie", "windenergie op zee", "zonne-energie"),
                 levels = niveaus),
  share = c(40, 35, 13, 3, 2, 7)
)
# leader_length is shortened from its 0.15 default: on this narrower
# canvas the default lines push their labels past the figure's edge
cpb_donut(energie2, fill = bron, y = share,
          label_style    = "leader",
          leader_length  = 0.08,
          label_accuracy = 0.01,
          palette        = "extended",
          colour_index   = c(8, 9, 5, 6, 12, 14, 15),
          title = "Energiemix, met lijnlabels")
```

5.9 Dot plot

```
termen <- c("Werken ontmoedigd", "Ongelijkheid", "Hard werken loont",
            "Leeftijd", "Geslacht", "Kinderen")
schatting <- tibble(
  term = factor(termen, levels = termen),
  blok = factor(rep(c("Percepties", "Achtergrond"), each = 3),
                levels = c("Percepties", "Achtergrond")),
  coef = c(3.2, 2.9, -2.0, 3.5, 0.4, -1.4),
  lo   = c(-0.3, 2.1, -3.0, 2.6, -1.3, -3.3),
  hi   = c(6.6, 3.6, -1.1, 4.3, 2.2, 0.6)
)
cpb_dot(schatting, x = term, y = coef, lower = lo, upper = hi,
        group = blok, legend = "filled-empty",
```

```
        title = "Samenhang met voorkeurstarieven",
        xlab  = "%punt verandering")
```

5.10 Scatter plot

```
hh <- tibble(inkomen = round(rlnorm(400, log(2500), 0.35))) |>
  mutate(energierekening = round(90 + 0.04 * inkomen + rnorm(n(), 0, 35)),
         koopkracht      = round(rnorm(n(), (inkomen - 2500) / 1500, 2), 1))
cpb_scatter(hh, x = inkomen, y = energierekening,
            colour = cpb_cut(koopkracht, c(-Inf, -2, -1, 0, 1, 2, Inf),
                             labeller = label_pct_nl()),
            palette = "blues",
            legend_ncol = 2,
            title = "Energierekening naar inkomen",
            ylab  = "energierekening (euro per maand)",
            xlab  = "besteedbaar inkomen (euro per maand)",
            x_labels = label_euro_nl(),
            legend_title = "koopkracht (%)")
```

5.11 Histogram

```
duur <- tibble(maanden = round(rgamma(1200, 8, 0.6)))
cpb_hist(duur, x = maanden, binwidth = 2,
         title = "Verdeling van de duur", ylab = "aantal",
         xlab  = "duur (maanden)")
```

5.12 Classed map

```
gemeenten <- tibble(code = unique(cpb_nl_geo("gemeente")$code)) |>
  mutate(aandeel = runif(n(), 5, 75),
         klasse  = cpb_cut(aandeel, breaks = c(0, 20, 30, 40, 50, 60, Inf),
                           labeller = label_pct_nl()))
cpb_map(gemeenten, region = code, value = klasse,
        palette = "blues",
        title = "Aandeel huishoudens met zonnepanelen",
        legend_title = "aandeel")
```